

Microshift edge Deployment

Prerequisites:

- System requirements.

Parameter	Minimal Required Value
System type	A dedicated host or virtual machine. Note that RHEL image builder is not supported in containers, including Red Hat Universal Base Images (UBI).
Processor	2 cores
Memory	4 GiB
Disk space	20 GiB of free space in the <code>/var/cache/</code> filesystem
Access privileges	root
Network	Internet connectivity to the Red Hat Content Delivery Network (CDN).

- Required packages `<osbuild-composer composer-cli cockpit-composer >`

Prodecule:

S1: Enable the following

```
systemctl enable --now osbuild-composer.socket
systemctl enable --now cockpit.socket
```

S2: source the following for auto command completion

```
source /etc/bash_completion.d/composer-cli
```

S3: Verify status

```
composer-cli status show
```

S4: RHEL image builder osbuild-composer back end does not inherit the system repositories located in the /etc/yum.repos.d/ directory. Instead, it has its own set of official repositories defined in the /usr/share/osbuild-composer/repositories directory. So (COPY) cp All the repo files from the /usr/share/osbuild-composer/repositories to /etc/osbuild-composer/repositories/ If /etc/osbuild-composer/repositories/ is not present create it.

```
cp /usr/share/osbuild-composer/repositories/rhel*
/etc/osbuild-composer/repositories/
systemctl restart osbuild-composer
```

S5: enable RHOCP and fast-patch repo

```
sudo subscription-manager repos \
--enable=rhocp-4.20-for-rhel-9-$(uname -i)-rpms \
--enable=fast-datapath-for-rhel-9-$(uname -i)-rpms
```

S6: for this lab we will be using RHSM (Redhat subscription manager) for repo source. Appstream and baseos will come default for osbuild-composer we need to manually specify any other repos or sources,(osbuild-composer uses toml format files.)

```
cat /tmp/microshift.toml
id = "microshift"
name = "Red Hat build of MicroShift"
type = "yum-baseurl"
url = "https://cdn.redhat.com/content/dist/layered/rhel9/x86_64/rhocp/4.21/os"
check_gpg = false
check_ssl = false
check_repogpg = false
rhsm = true
```

```
cat /tmp/fastdatapatch.toml
id = "fastdatapatch"
name = "Red Hat build of fastdatapatch"
type = "yum-baseurl"
url =
"https://cdn.redhat.com/content/dist/layered/rhel9/x86_64/fast-datapath/os"
check_gpg = false
```

```
check_ssl = false
check_repogpg = false
rhsm = true
```

Above are the 2 toml files which tells osbuild to use redhat CDN as its source and use RHSM to get access of the mentioned 2 repos (NOTE: These 2 repos must be enabled) fast-datapath-for-rhel-9-x86_64-rpms and rhocp-4.21-for-rhel-9-x86_64-rpms

To add the two sources to osbuild perform the following

```
composer-cli sources add /tmp/microshift.toml
```

```
composer-cli sources add /tmp/fastdatapatch.toml
```

Post adding the two sources Run: `composer-cli sources list`

```
[root@jaya ~]# composer-cli sources list
appstream
baseos
fastdatapatch
microshift
```

S7: systemctl restart osbuild-composer

S8: before starting we need to make sure that worker is properly configured. To configure the worker we need to edit `cat /usr/lib/systemd/system/osbuild-worker@.service` file

We have to add `ExecStart=/usr/libexec/osbuild-composer/osbuild-worker -unix`

`/run/osbuild-composer/job.socket` in the file so that worker doesn't have to re-register again after reboot osbuild and to make sure that worker only uses -unix for communication not http as it might result in errors while building image/booting image or an iso

```
[root@jaya osbuild-composer]# cat /usr/lib/systemd/system/osbuild-worker@.service
[Unit]
Description=OSBuild Composer Worker (%i)
Requires=osbuild-local-worker.socket
After=network.target osbuild-local-worker.socket

[Service]
Type=simple
PrivateTmp=true
ExecStart=/usr/libexec/osbuild-composer/osbuild-worker -unix /run/osbuild-composer/job.socket
Restart=always
RestartSec=1s
CPUSchedulingPolicy=batch
IOSchedulingClass=idle
CacheDirectory=osbuild-worker
# systemd >= 240 sets this, but osbuild-worker runs on earlier versions
Environment="CACHE_DIRECTORY=/var/cache/osbuild-worker"

[Install]
WantedBy=osbuild-composer.service
[root@jaya osbuild-composer]#
```

S9: You can create a blueprint and image in both cli and webui we will be using cli for this lab.

Create a blueprint.toml

```
name = "Microshift_test5"
description = "This is test5 "
version = "0.0.0"
modules = []
enabled_modules = []
groups = []
```

```

[[packages]]
name = "microshift"

[[packages]]
name = "microshift-networking"

[[packages]]
name = "microshift-release-info"

[[packages]]
name = "microshift-selinux"

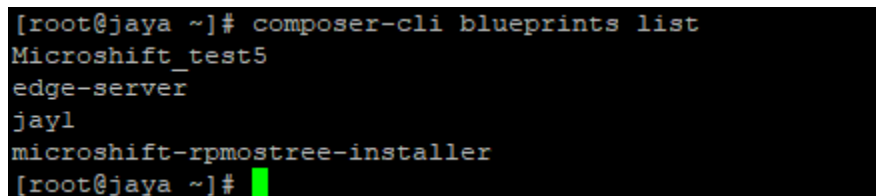
[customizations]
hostname = "microshift.jaya.co"
installation_device = "/dev/sda1"

[[customizations.user]]
name = "spaz"
password =
"$6$WrHOj3WRBKZLCluy$OPJEFxFyQPpJzj40mWSq8t8KRTvlo51cmgyan2R7L4yRx8feI9BLCw7mtaWYoU3G8
JrmVTYxH1.9lVGUbwsav/"
key = "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQgQDk4lFfHPWOXgRbbGP5OiwujYXvMKtLlUrVEKJZfLwITV+7UhQ8Fz5YJF
5nmHkIdZnjCwf9fCfQAvnjpeSq2Op3fFvl35JYv1T/t6EA+hyrTeZa3zWgW8oL8UzdaTAQamuI72n8CCWCJTBY
qKoP/+wiPqTk9tNcVivVsWAFBE5DDCQ8f+FMmDSGG5JRbPm0y96IcKbpy37+kCg5UhrXYWotAhHdbEfmOmARJ5
lyZlnYvxvTUnVaOseVCh2qeiYu7UAL+uVUpWTLGYWIr8YWdpceMyT9LY17C1/RpAln9Y5g0gQwdn7VP/c676
efWtvtTf9lmlDr99pBC6d264k6xJCTj8Od2/Is4vMfP02rGufA/gEs8tRgvg+NvmAPSD7LHFZtwcq2Cw0It2Qa
sFggPrFw9JShzxh6klfqO8nxa8G22s+5PIDZYtoXZCZqWIjFXEYzAkPhr2Na+At82p4uForuqhMoPzm9GlgEY
ZbRmOFjpiSc7znizPsKDMCc4Qjc= spaz@jaya.works.co"
groups = ["wheel"]
[customizations.firewall]
ports = ["6443:tcp"]
[customizations.firewall.services]
enabled = ["http", "https", "ntp", "dhcp", "ssh"]
disabled = ["telnet"]
[customizations.services]
enabled = ["sshd", "microshift"]

```

S10: Then push the blueprint

```
composer-cli blueprints push edge.toml
```



```

[root@jaya ~]# composer-cli blueprints list
Microshift_test5
edge-server
jayl
microshift-rpmootree-installer
[root@jaya ~]#

```

S11: First we will start edge-commit as it is required for creating ostree repo

```

composer-cli compose start Microshift_test5 edge-commit
composer-cli compose status

```

```
[root@jaya ~]# composer-cli compose status
```

ID	Status	Time	Blueprint	Version	Type	Size
f19040aa-a45b-426d-b4bd-6978a4df1cf4	FINISHED	Sun Jun 7 22:26:50 2026	edge-server	0.0.1	edge-commit	
435dcf16-3069-4000-9a62-4de34078e003	FINISHED	Mon Jun 8 13:42:19 2026	microshift-rpmostree-installer	0.0.1	edge-installer	
175c4b92-8291-49e8-a526-6004e8dee52f	FAILED	Sun Jun 7 22:11:37 2026	edge-server	0.0.1	qcow2	

```
[root@jaya ~]#
```

S12: Once we see finished in the status output we can download the tar file (image)

```
composer-cli compose image f19040aa-a45b-426d-b4bd-6978a4df1cf4
```

This will download an tar file which we need to extract

```
tar xvf f19040aa-a45b-426d-b4bd-6978a4df1cf4-commit.tar
```

S13: We need to host a container to act as our ostree repository, we can use the hostos for the activity but for the lab we will be using an nginx container running on rhel 9 image

Create an nginx file with following entries

```
events {
}

http {
    server {
        listen 8080;
        root /usr/share/nginx/html;

        # Enables file directory listing for your repo
        location / {
            autoindex on;
        }
    }
}

pid /run/nginx.pid;
daemon off;
```

Then we will create an Containerfile

```
FROM registry.access.redhat.com/ubi9/ubi
RUN yum -y install nginx && yum clean all
COPY repo /usr/share/nginx/html/repo/
COPY nginx /etc/nginx/nginx.conf
EXPOSE 8080
CMD ["/usr/sbin/nginx", "-c", "/etc/nginx/nginx.conf"]
ARG commit
ADD ${commit} /usr/share/nginx/html/
```

Build the container image

```
podman build -t microshift-image-0.0.1 --build-arg
commit=f19040aa-a45b-426d-b4bd-6978a4df1cf4-commit.tar .
```

Run the container

```
podman run --name rpm-ostree-repository --rm -d -p 8080:8080
localhost/microshift-image-0.0.1:latest
```

S14: Create another blueprint

microshift-rpmostree-installer.toml

```
name = "microshift-rpmostree-installer"
description = ""
version = "0.0.1"
modules = []
groups = []
```

Push the blueprint

```
composer-cli blueprints push microshift-rpmostree-installer.toml
```

Start building the edge-installer

```
composer-cli compose start-ostree --ref rhel/9/x86_64/edge --url
http://localhost:8080/repo microshift-rpmostree-installer edge-installer
```

S15: Create a [kickstart.ks](#) file which will be used to setup microshift

```
lang en_US.UTF-8
keyboard us
timezone UTC
zerombr
clearpart --all --initlabel
part /boot/efi --fstype=efi --size=200
part /boot --fstype=xfs --asprimary --size=800
part pv.01 --grow
volgroup rhel pv.01
logvol / --vgname=rhel --fstype=xfs --size=10000 --name=root
reboot
text
network --bootproto=dhcp
ostreesetup --nogpg --url=http://172.24.229.201:8080/repo/ --osname=rhel --remote=edge
--ref=rhel/9/x86_64/edge

%post
# Add the pull secret to CRI-O and set root user-only read/write permissions
cat > /etc/crio/openshift-pull-secret << EOF
<YOUR_PULL_SECRET_JSON_FORMAT>
EOF
chmod 600 /etc/crio/openshift-pull-secret
%end
%post
# Configure the firewall with the mandatory rules for MicroShift
firewall-offline-cmd --zone=trusted --add-source=10.42.0.0/16
firewall-offline-cmd --zone=trusted --add-source=169.254.169.1
%end
```

Combine both iso and [kickstart.ks](#) (sudo dnf install lorax)

```
mkksiso kickstart.ks 9509b4de-dc66-4020-9d24-86476ffe086f-installer.iso
microshift-rpmostree-installer.iso
```

Post deployment verification:

```
[root@microshift ~]# oc get no
NAME                STATUS    ROLES                                AGE     VERSION
microshift.jaya.co  Ready    control-plane, master, worker    3h6m    v1.34.7
[root@microshift ~]#
```

```
[root@microshift ~]# rpm-ostree status
State: idle
Deployments:
* edge:rhel/9/x86_64/edge
  Version: 9.7 (2026-06-08T06:46:06Z)
  Commit: 8a896aa5e9401f34ab9cb7f145c8c36ce4adf56f8ee3df8657f98d0d910e7234
[root@microshift ~]#
```